

Data Structures 1 – Winter 2025

Assignment A1

General Rules

Item	Details
Deadline	November 16, 11:59 p.m.
Late Penalty	10 pts if submitted Nov 17, 20 pts if submitted Nov 18, 30 pts if submitted Nov 19. No submissions accepted after Nov 20
Group Size	Pairs (two students)
Oral Defense	You may be asked to explain your solution orally. Failure to answer satisfactorily implies a grade 0 in each exercise involved in the discussion.
Academic Integrity	Any detected plagiarism or copying will be reported to the UG office.
Questions	None allowed during the exam period.
Language & Environment	Java (must compile in VS Code and pass all provided tests).
Deliverables	One zip file containing all Java source files (<i>wet part</i>). One PDF file with all written answers (<i>dry part</i>).

1. Big-O Simplification – 5 pts (Dry)

Simplify each expression to **Big-O notation**. Justify your answer using the definition of Big-O and/or its properties.

1. $O(5n^3 + 3n^2 + 100n + 1000) + O(n \cdot \log n)$
2. $O(2^n) \cdot O(n^2) + O(3^n)$
3. $O(\log_2 n) \cdot O(\sqrt{n}) + O(n \cdot \log n)$
4. $O(n!) + O(2^n) \cdot O(n^3)$

2. Sorting Algorithm Analysis – 25 pts (Dry and Wet)

Consider the following pseudo-code:

```
SORT(A):
  set B[i] := 0 for every i between 0 and 9
  for i := 0 to |A| - 1 do
    index := A[i]
    B[index] := B[index] + 1
  k = 0
  for i = 0 to 9 do
    j = 0
    while j < B[i] do
      A[k] := i
      k := k + 1
      j := j + 1
```

- a. (2 pts, dry) State the **precondition** required for correctness.
 - b. (5 pts, dry) Determine the **worst-case time complexity**. Provide a detailed asymptotic analysis.
 - c. (12 pts, dry) Prove **correctness** using the **loop invariant**.
 - d. (3 pts, wet) Implement the algorithm in **Java**.
 - e. (3 pts, wet) Supply **meaningful unit tests**.
-

3. Palindrome Collector – 30 pts (Dry and Wet)

A **Palindrome Collector** is a sequence that can grow or shrink **only at its two ends** (a double-ended queue) and can always tell—in **constant time**—whether the current sequence is a **palindrome** (reads identically left-to-right and right-to-left).

After every successful insertion at either end the collector **does NOT rotate**; it simply updates its internal symmetry information.

Deletions likewise do **not** trigger any rearrangement.

- a. (5 pts, wet) (Define a Java interface `PalindromeCollector` of `Char` with operations `addFirst`, `addLast`, `removeFirst`, `removeLast`, `isPalindrome`, `isEmpty`, `size()`. Document **pre-/post-conditions** in clear English **without revealing any internal representation**
- b. (5 pts, dry) Design a concrete class `PalindromeCollectorImp` that stores chars in a **fixed-length circular array**. Declare **all fields** needed to guarantee **$O(1)$** time for **every** operation including `isPindrome` (hint: you may need to maintain some field to count the number of mismatches in the word compared to a palindrome).
- c. (10 pts, dry and wet) Provide a **representation invariant** **expressen** in English and in the form of boolean `repOK()` method in java and describe, in English, an **abstract function** that explains, in terms of the TAD, what the concrete state means.
- d. (7 pts, wet) Implement in java the **5 core procedures** (`addFirst`, `addLast`, `removeFirst`, `removeLast`, `isPalindrome`) maintaining $O(1)$.
- e. (3 pts, wet) Provide **unit tests** covering meaningful cases (e.g, empty collector, single-element collector, even-length palindrome, odd-length palindrome, non-palindrome after inserts, palindrome destroyed and restored by removals, etc).

4. Basic Blockchain System – 40 pts (wet)

Problem Description

Implement a **simplified blockchain (SB)** that acts as an **immutable distributed ledger**. The core is a **linked list of blocks**, each holding:

- A **fixed number of transactions** 固定数量的交易
- A **hash of the previous block** 前一个区块的哈希值

Participants, identified by string **addresses** (e.g., "Alice"), can hold **BB-coin**, the SB currency. Their **balance** is also maintained by the system. Initial **genesis block** gives **1000 BB** to address "0".

Core Components & Required Complexities

Component	Key Operations	Required Complexity
Blockchain	<code>processTransaction</code> , <code>addBlock</code> , <code>getBlock</code> , <code>size</code> , <code>getLastBlock</code> , <code>getBlocks</code> , <code>getBalance</code>	See below
Block	<code>addTransaction</code> , <code>getFirstTransaction</code> , <code>removeTransaction</code> , <code>isFull</code> , hash getters	$O(1)$ except removeTransaction in $O(T)$
Transaction	<code>Constructor</code> , <code>revert</code> , <code>getters</code>	$O(1)$
Balance	<code>getBalance(address)</code>	$O(A)$

Where: **T**: number of transactions stored in **one** block, **A**: number of distinct addresses that currently hold a balance, **n**: number of blocks already in the chain.

T: 单个区块中存储的交易数量
A: 当前拥有余额的不同地址数量
n: 链中已有的区块数量

Detailed Requirements

4.1 Interface Blockchain and class SBlockchain

Constructor

- `SBlockchain()` – creates genesis block with `previousHash = 0`, empty transaction list, and initial balance 1000 for address "0".

Methods

- `processTransaction(String from, String to, int amount)`
 – Adds transaction to current block.
 – If block becomes full, automatically calls `addBlock()`.
 – Throws `IllegalArgumentException` if `amount <= 0`.
 – **Complexity**: $O(T.A)$ when block becomes full; otherwise $O(1)$.
 - `addBlock()`
 – Appends current block to chain and **executes** all transactions.
 – Failed transactions are **marked reverted**; balances **unchanged**.
 – **Complexity**: $O(T.A)$.
 - `getBlock(int index)` – $O(n)$ worst-case.
 - `size()` – $O(1)$.
 - `getLastBlock()` – $O(1)$.
 - `getBlocks()` – $O(1)$ return of list of blocks.
 - `getBalance(String address)` – $O(A)$.

Extra method

boolean `repOK()` – validates chain integrity:

1. Balances equal sum of **non-reverted** transactions.
2. previousHash of block i equals currentHash of block $i-1$.
3. All blocks contain **same number** of transactions.
4. Block numbers **strictly increasing**.

State the **computational complexity** of `repOK()`.

4.2 Interface Balance and class BalanceImp

- `int getBalance(String address)` – returns balance or 0 if absent.
Must be $O(A)$.

4.3 Class Block

Fields: transactions, block number, blockHash, previousHash (int counters).
Maintain **insertion order**.

Constructor

- `Block(int previousHash, int transactionsPerBlock, int blockNumber)` – blockHash = previousHash + 1.

Operations

- `addTransaction(Transaction t)` – $O(1)$.
- `getFirstTransaction()` – $O(1)$.
- `removeTransaction(int index)` – $O(T)$ and preserves order.
- `isFull()` – $O(1)$.
- `getBlockHash()` / `getPreviousHash()` – $O(1)$.

Extra method

- `repOK()` – true iff previousHash + 1 == currentHash.

4.4 Class Transaction

Constructor

- `Transaction(String from, String to, int amount)` – throws if amount ≤ 0 or addresses invalid.

Operations

- `revertTransaction()` – marks transaction reverted.
- Getters: `getFromAddress()`, `getToAddress()`, `getAmount()`, `reverted()`

Implementation Notes

- See general rules.
- **Do NOT use** Java's built-in classes (e.g. Queue, Stack, or LinkedList).
- Provide **your own data structures**. You can use all code we used in the tutorials 1,2 and 3.
- Supply **meaningful unit tests** covering edge cases.
- Justify **complexity analysis** for every method.
- Code must be **readable, correct, and pass all supplied tests**.